Chapter 1

Introduction to Parallel and Distributed Computing

1.1 Learning Objectives

By the end of this chapter, students should be able to:

- Explain the motivation for parallel and distributed computing
- Distinguish between sequential, parallel, and distributed systems
- Describe real-world applications of parallel computing
- Classify architectures using Flynn's Taxonomy
- Compute speedup and efficiency for parallel programs

1.2 Introduction

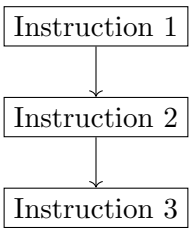
As computational problems grow in scale and complexity, traditional sequential execution models become insufficient. Applications such as large-scale data analytics, artificial intelligence, and scientific simulations require significantly more computational power than a single processor can provide.

Parallel and Distributed Computing addresses this challenge by enabling multiple processing units to cooperate in solving computational problems.

1.3 Sequential, Parallel, and Distributed Computing

1.3.1 Sequential Computing

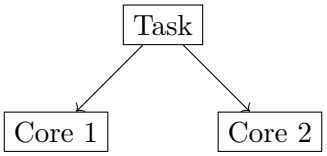
Sequential computing executes instructions one after another in a single processing unit.



Limitation: Execution speed is constrained by a single processing unit.

1.3.2 Parallel Computing

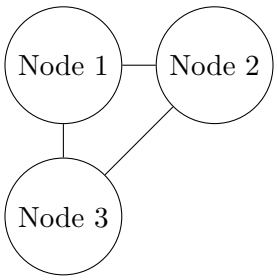
Parallel computing divides a problem into smaller tasks that can be executed simultaneously across multiple processors or cores.



Key Idea: Improve performance through simultaneous execution.

1.3.3 Distributed Computing

Distributed computing consists of multiple independent computers (nodes) connected via a network, each with its own memory.



Key Idea: Computation is distributed across multiple machines that communicate via message passing.

1.4 Motivation for Parallel Computing

1.4.1 Performance Improvement

Parallel systems can significantly reduce execution time for computationally intensive tasks.

1.4.2 Scalability

Systems can be scaled by adding more processors or nodes to handle larger workloads.

1.4.3 Energy Efficiency

Multiple smaller cores may provide better energy efficiency than a single high-performance core.

1.5 Applications of Parallel and Distributed Computing

Parallel and distributed computing is widely used in:

- Artificial Intelligence and Machine Learning
- Scientific simulations (e.g., weather prediction, physics simulations)
- Financial modeling and risk analysis
- Cryptography and blockchain systems
- Cloud computing and large-scale web services

1.6 Flynn's Taxonomy

Flynn's Taxonomy classifies computer architectures based on the number of instruction and data streams.

1.6.1 SISD (Single Instruction, Single Data)

- Traditional sequential computing model
- One instruction operates on one data element at a time

1.6.2 SIMD (Single Instruction, Multiple Data)

- A single instruction operates on multiple data elements simultaneously
- Common in GPUs and vector processors

1.6.3 MISD (Multiple Instruction, Single Data)

- Multiple instructions operate on the same data stream
- Rarely used in practical systems

1.6.4 MIMD (Multiple Instruction, Multiple Data)

- Multiple processors execute different instructions on different data
- Most modern multi-core and distributed systems fall under this category

1.7 Performance Metrics

1.7.1 Speedup

Speedup measures the improvement in execution time achieved by parallelization:

$$S = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$$

Example: If a program takes 10 seconds sequentially and 2 seconds in parallel, then $S = 5$.

1.7.2 Efficiency

Efficiency measures how effectively processing resources are utilized:

$$E = \frac{S}{P}$$

where P is the number of processors.
Example: If $S = 5$ and $P = 8$, then $E = 0.625$.

1.8 Amdahl's Law

Amdahl's Law provides the theoretical limit of speedup based on the fraction of the program that can be parallelized:

$$S_{\max} = \frac{1}{(1-f) + \frac{f}{P}}$$

where:

- f is the parallelizable fraction of the program
- P is the number of processors

Key Insight: The non-parallelizable portion of a program limits overall speedup.

1.9 Challenges in Parallel Computing

- Communication overhead between processors
- Synchronization and coordination issues
- Load imbalance among processors
- Increased complexity in debugging and testing

1.10 Summary

- Parallel computing improves performance by executing tasks simultaneously
- Distributed systems involve multiple independent machines communicating over a network
- Flynn's Taxonomy classifies computing architectures
- Speedup and efficiency are key performance metrics
- Amdahl's Law limits achievable parallel performance

1.11 Exercises

1. Explain the differences between sequential, parallel, and distributed computing.
2. Provide two real-world applications of parallel computing.
3. Classify the following using Flynn's Taxonomy:
    - GPU
    - Single-core CPU
    - Multi-core CPU
4. A program takes 20 seconds sequentially and 5 seconds in parallel. Compute:
    - Speedup
    - Efficiency (for 4 processors)
5. If 80% of a program can be parallelized, compute the maximum speedup using 8 processors.